# Set 23: Modifying DataFrames



## Skill 23.1: Add columns to a DataFrame

**Skill 23.1 Concepts**

In the previous lesson, we learned what a DataFrame is and how to select subsets of data from one. In addition to selecting data, sometimes we want to add new information to a DataFrame. One way to add data is to add a new column.

Suppose we own a hardware store called The Handy Woman and have a DataFrame containing inventory information,

**Code**

```
import pandas as pd
df = pd.read_csv("TheHandyWoman.csv", delimiter = "\t")
display(df)
```

**Output**

| | Product ID | Product Description | Cost to Manufacture | Price |
|---|---|---|---|---|
| 0 | 1 | 3 inch screw | 0.5 | 0.75 |
| 1 | 2 | 2 inch nail | 0.1 | 0.25 |
| 2 | 3 | hammer | 3.0 | 5.50 |
| 3 | 4 | screwdriver | 2.5 | 3.00 |

It looks like the actual quantity of each product in our warehouse is missing! The following code can be used to add that information to our DataFrame.

**Code**

```
df['Quantity'] = [100, 150, 50, 35]
display(df)
```

**Output**

| | Product ID | Product Description | Cost to Manufacture | Price | Quantity |
|---|---|---|---|---|---|
| 0 | 1 | 3 inch screw | 0.5 | 0.75 | 100 |
| 1 | 2 | 2 inch nail | 0.1 | 0.25 | 150 |
| 2 | 3 | hammer | 3.0 | 5.50 | 50 |
| 3 | 4 | screwdriver | 2.5 | 3.00 | 35 |

In the example above, we added a new column by providing a list of the same length as the existing DataFrame.

We can also add a new column that is the same for all rows in the DataFrame. Let's return to our inventory example

**Code**

```
df['In Stock?'] = True

display(df)
```

**Output**

| | Product ID | Product Description | Cost to Manufacture | Price | Quantity | In Stock? |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 inch screw | 0.5 | 0.75 | 100 | True |
| 1 | 2 | 2 inch nail | 0.1 | 0.25 | 150 | True |
| 2 | 3 | hammer | 3.0 | 5.50 | 50 | True |
| 3 | 4 | screwdriver | 2.5 | 3.00 | 35 | True |

Finally, you can add a new column by performing a function on the existing columns.

Maybe we want to add a column to our inventory table with the amount of sales tax that we need to charge for each item. The following code multiplies each Price by 0.075, the sales tax for our state,

**Code**

```
df['Sales Tax'] = df.Price * 0.075

display(df)
```

**Output**

| | Product ID | Product Description | Cost to Manufacture | Price | Quantity | In Stock? | Sales Tax |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 3 inch screw | 0.5 | 0.75 | 100 | True | 0.05625 |
| 1 | 2 | 2 inch nail | 0.1 | 0.25 | 150 | True | 0.01875 |
| 2 | 3 | hammer | 3.0 | 5.50 | 50 | True | 0.41250 |
| 3 | 4 | screwdriver | 2.5 | 3.00 | 35 | True | 0.22500 |

**Skill 23.1 Exercise 1**

## Skill 23.2: Perform Column Operations

**Skill 23.2 Concepts**

In the previous exercise, we learned how to add columns to a DataFrame.

Often, the column that we want to add is related to existing columns, but requires a calculation more complex than multiplication or addition.

For example, imagine that we have the following table of customers.

| | Name | Email |
|---|---|---|
| 0 | JOHN SMITH | john.smith@gmail.com |
| 1 | Jane Doe | jdoe@yahoo.com |
| 2 | joe schmo | joeschmo@hotmail.com |

It's a little annoying that the capitalization is different for each row. Perhaps we'd like to make it more consistent by making all of the letters uppercase.

We can use the apply function to apply a function to every value in a particular column. For example, this code overwrites the existing 'Name' columns by applying the function upper to every row in 'Name'.

**Code**

```
df = pd.read_csv("Customers.csv", delimiter = "\t")
df['Name'] = df['Name'].apply(str.upper)
display(df)
```

**Output**

| | Name | Email |
|---|---|---|
| 0 | JOHN SMITH | john.smith@gmail.com |
| 1 | JANE DOE | jdoe@yahoo.com |
| 2 | JOE SCHMO | joeschmo@hotmail.com |

**Skill 23.2 Exercise 1**

## Skill 23.3: Interpret lambda functions

**Skill 23.3 Concepts**

A *function* is an object that is able to accept some sort of input, possibly modify it, and return some sort of output. In Python, a *lambda function* is a one-line shorthand for function. A simple lambda function might look like this,

**Code**

```
add_two = lambda my_input: my_input + 2
print(add_two(3))
print(add_two(100))
print(add_two(-2))
```

**Output**

```
5
102
0
```

Below breaks this syntax down:

1. The function is stored in a variable called *add_two*
2. *lambda* declares that this is a lambda function (This is similar to how we use def to declare a function)
3. *my_input* is what we call the input we are passing into *add_two*
4. We are returning *my_input + 2* (with normal Python functions, we use the keyword return)

Lambda functions can also be applied to strings. For example, the following checks if a string is a substring of the string "This is the master string",

**Code**

```
is_substring = lambda my_string: my_string in "This is the master string"


print(is_substring('I'))
print(is_substring('am'))
print(is_substring('the'))
print(is_substring('master'))
```

**Output**

```
False
False
True
True
```

If statements can also be included in lambda functions. Below is the format of a lambda function with if statements.

```
<WHAT TO RETURN IF STATEMENT IS TRUE> if <IF STATEMENT> else <WHAT TO RETURN IF
STATEMENT IS FALSE>
```

Below is an example that checks whether or not someone got an 'A',

**Code**

```
check_if_A_grade = lambda grade: 'Got an A!' if grade >= 90 else 'Did not get an A...'
print(check_if_A_grade(89))
```

**Output**

```
Did not get an A...
```

Lambda functions only work if we're just doing a one-line command. If we wanted to write something longer, we'd need a more complex function. Lambda functions are great when you need to use a function once. Because you aren't defining a function, the reusability aspect is not present with lambda functions.

Lambda functions allow us to efficiently run an expression and produce an output for a specific task, such as defining a column in a table, or populating information in a dictionary without defining a separate function the traditional way.

**Skill 23.3 Exercise 1**

## Skill 23.4: Apply a lambda function to a column

### Skill 23.4 Concepts

In Pandas, we often use lambda functions to perform complex operations on columns. For example, suppose that we want to create a column containing the email provider for each email address in the following table,

| | Name | Email |
|---|---|---|
| 0 | JOHN SMITH | john.smith@gmail.com |
| 1 | JANE DOE | jdoe@yahoo.com |
| 2 | JOE SCHMO | joeschmo@hotmail.com |

The following example illustrates how to do this with a lambda function,

**Code**

```
provider = lambda e: e.split("@")[1]
df['Provider'] = df['Email'].apply(provider)
display(df)
```

**Output**

| | Name | Email | Provider |
|---|---|---|---|
| 0 | JOHN SMITH | john.smith@gmail.com | gmail.com |
| 1 | JANE DOE | jdoe@yahoo.com | yahoo.com |
| 2 | JOE SCHMO | joeschmo@hotmail.com | hotmail.com |

**Skill 23.4 Exercise 1**

## Skill 23.5: Rename columns

**Skill 23.5 Concepts**

When we get our data from other sources, we often want to change the column names. For example, we might want all of the column names to follow variable name rules, so that we can use *df.column_name* (which tab-completes) rather than *df['column_name']* (which takes up extra space).

You can change all the column names at once by setting the columns property to a different list. This is great when you need to change all the column names at once, but be careful! You can easily mislabel columns if you get the ordering wrong. Here's an example,

**Code**

```
df = pd.DataFrame({
    'name': ['John', 'Jane', 'Sue', 'Fred'],
    'age': [23, 29, 21, 18]
})
df.columns = ['First Name', 'Age']
display(df)
```

**Output**

| | First Name | Age |
|---|---|---|
| 0 | John | 23 |
| 1 | Jane | 29 |
| 2 | Sue | 21 |
| 3 | Fred | 18 |

You also can rename individual columns by using the *rename* method. The rename method accepts a dictionary as an argument. Below is an example,

**Code**

```
df = pd.DataFrame({
    'name': ['John', 'Jane', 'Sue', 'Fred'],
    'age': [23, 29, 21, 18]
})
df.rename(columns={
    'name': 'First Name',
    'age': 'Age'},
    inplace=True)
```

**Output**

| | First Name | Age |
|---|---|---|
| 0 | John | 23 |
| 1 | Jane | 29 |
| 2 | Sue | 21 |
| 3 | Fred | 18 |

The code above will rename *name* to *First Name* and *age* to *Age*.

Using rename with only the columns keyword will create a new DataFrame, leaving your original DataFrame unchanged. Passing in the keyword argument *inplace=True* lets us edit the original DataFrame.

There are several reasons why .rename is preferable to the columns property,

- You can rename just one column
- You can be specific about which column names are getting changed (with .column you can accidentally switch column names if you're not careful)

**Note:** If you misspell one of the original column names, this command won't fail. It just won't change anything.

**Skill 23.5 Exercises 1 and 2**